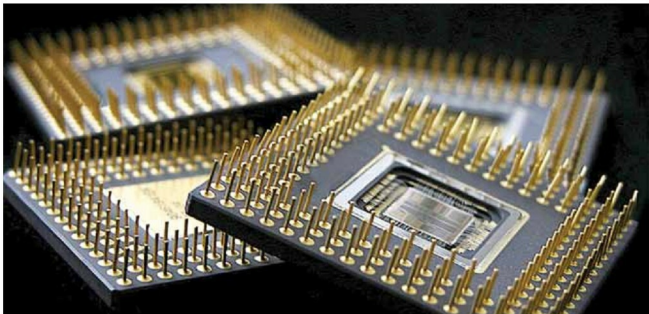


Operating Modes in x86 Systems: An Inside Story

Here's a discussion on operating (or addressing) modes in x86 architecture, and on segment selectors, descriptors, paging and multitasking (rather, TSS). Readers trying to program kernels/bootloaders, or thinking of writing their own toy OS, will find it interesting. Examples are limited to Intel's IA-32 architecture only.



The firmware in the ROM that runs first when an x86-processor is powered up, which is referred to as the BIOS, does a series of primitive operations. The mode in which the BIOS runs is the *real mode* or the *real addressing mode*. An x86 system would typically run in either real or protected mode. When the system is reset or powered up, it is first initialised in real mode and will remain in this mode unless it is switched to the protected mode by the software.

So what exactly is real addressing mode?

As mentioned earlier, when an x86 system is powered up or reset, it is in real mode. If you wondered why this is so, it's because in the real addressing mode, an x86 system becomes backward compatible with all previous x86 chips, and thus allows the BIOS and other prehistoric software to run. 8086, the notable 16-bit microprocessor from Intel and the very first with the x86-architecture, while implementing *memory segmentation*, operates only in real mode. Software, in real mode, can directly access all memory, I/O addresses and hardware peripherals; however, the memory that can be addressed is limited to only 2^{20} bytes, i.e., only 1 MB of memory can be addressed. Further, the entire 1 MB of memory cannot be active at the same time but partitions of 64 KB segments can. Each segment has its own starting address and is numbered from 00000H to FFFFFH.

Because paging is not active in real mode, the linear addresses generated are the same as the physical addresses. Segment registers hold the starting address of the active segments in the entire memory. Refer to Figure 1 for clarity.

In protected mode, nevertheless, memory larger than 1 MB can be accessed. All modern operating systems work in protected mode and provide advanced features like paging, virtual memory and multi-tasking, and thus have a more controlled environment.

We can now focus on the next level—*protected mode*, or even better, *protected virtual address mode*.

Protected mode: Unlock the possibilities

X86 systems are always powered up/ reset in real mode. But setting bit 0 (Protected Mode Enable bit) in the control register CR0 makes it possible to start operations in the protected mode.

Here is the 'bouquet of benefits' that protected mode provides:

- **Larger physical address space** - Protected mode seamlessly increases the addressable physical address space. For example, a 32-bit x86 system can address up to 64 GB of RAM in the *Physical Address Extension (PAE)* mode against 4 GB (i.e., 2^{32} bytes) of RAM without PAE. The PAE feature adds 4 extra bits to a standard 32-bit system (so memory becomes addressable up to 2^{36} bytes). Modern 32-bit OSs like Linux take advantage of the PAE feature to support full 64 GB of